

Machine Learning:

Classifier Evaluation

Xiuxia Du, Ph.D.

Department of Bioinformatics and Genomics
University of North Carolina at Charlotte

Outline

- Why do we need to evaluate classifiers?
- Contingency table and related core metrics
 - Accuracy, precision, recall, specificity
 - F1 score
- ROC and AUC
- Precision-Recall curve
- Estimation of generalization performance
- Overfitting

Why Do We Need to Evaluate Classifiers?

- **Training performance is misleading:** A model can memorize training data (overfitting) and fail on new data.
- **Different mistakes have different consequences:** False negatives (missed positives) vs false positives (false alarms) matter differently by application.
- **Accuracy alone can be dangerously wrong:** On imbalanced data, a useless model can still achieve very high accuracy.

Why Do We Need to Evaluate Classifiers?

- **Classifiers require a decision threshold:** Probabilities must be converted into labels; threshold choice controls the precision–recall tradeoff.
- **We must compare models fairly:** Evaluation provides a consistent basis to choose the best model for the task.
- **Scientific and real-world accountability:** Metrics, confusion matrices, ROC/PR curves, and cross-validation justify model reliability.

Bottom line: Evaluation tells us how a classifier will behave on unseen data and whether its errors match the real-world goal.

Contingency Table and Related Metrics

From class probability to class label

- Most classifiers output a score or probability:

$$\hat{p}(y = 1|x)$$

- To turn that into a class label, we choose a threshold t :

$$y = \begin{cases} 1 & \text{if } \hat{p} \geq t \\ 0 & \text{otherwise} \end{cases}$$

- Once we choose the threshold t , we can construct a 2×2 **contingency table**.

Contingency table

- Also known as a *confusion matrix* or an *error matrix*.
- Binary classification

	Predicted Negative (0)	Predicted Positive (1)
Actual Negative (0)	TN	FP
Actual Positive (1)	FN	TP

- Key Idea:
 - TN = correctly predicted negatives
 - FP = false alarms
 - FN = missed positives
 - TP = correctly predicted positives

Contingency table

- From these four numbers come a number of metrics:
 - sensitivity, recall
 - specificity
 - precision
 - accuracy
 - F_1 score
 - and more
- **A confusion matrix describes classifier behavior at one particular decision threshold.**
- As we change the probability threshold t , the contingency table changes and these metrics change too.

Core Metrics

	Pred. 0	Pred. 1	Total	Metric	Metric
Act. 0	TN	FP	TN+FP	$Sp = \frac{TN}{TN + FP}$	$FPR = \frac{FP}{FP + TN}$
Act. 1	FN	TP	FN+TP	$R = Se = TPR = \frac{TP}{TP + FN}$	
Total	TN+FN	FP+TP	TP+TN+FP+FN	$A = \frac{TP + TN}{TP + TN + FP + FN}$	
Metric		$P = \frac{TP}{TP + FP}$		$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	
Metric		$FDR = \frac{FP}{FP + TP}$			

Sp = Specificity; Se = Sensitivity; R = Recall; P = Precision; A = Accuracy

Core Metrics

- **Accuracy**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

[V] Good when classes are balanced.

[X] Bad when classes are imbalanced. A 98%-negative dataset allows 98% accuracy by predicting everyone negative.

- **Precision (Positive Predictive Value)**

$$\text{Precision} = \frac{TP}{TP + FP}$$

Interpretation: When the model predicts positive, how often is it right?

Core Metrics

- **Recall (Sensitivity or True Positive Rate)**

$$\text{Recall} = \frac{TP}{TP + FN}$$

Interpretation: Out of all true positives, what fraction did we detect? High sensitivity means few false negatives. Important in screening and rare-event detection.

- **Specificity (True Negative Rate)**

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Interpretation: Out of all true negative, what fraction did we correctly reject? High specificity means few false positives.

Core Metrics

- **F1 Score**

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- To get a good F1 score, both the precision and recall need to be high.
- F1 Score is especially useful in imbalanced datasets where a low value in either precision or recall can significantly lower the F1 score.

Core Metrics

- Specificity = True Negative Rate (TNR)
- False Positive Rate (FPR)

$$FPR = \frac{FP}{FP + TN} = 1 - \frac{TN}{FP + TN} = 1 - \text{Specificity}$$

- True Positive Rate (TPR) = Recall = Sensitivity
- False Discovery Rate (FDR)

$$FDR = \frac{FP}{FP + TP} = 1 - \text{Precision}$$

ROC and AUC

ROC

- As the threshold t is changed:

$$1.0 \rightarrow 0.9 \rightarrow 0.8 \rightarrow \dots \rightarrow 0.2 \rightarrow 0.1 \rightarrow 0.0$$

we calculate the following at each t :

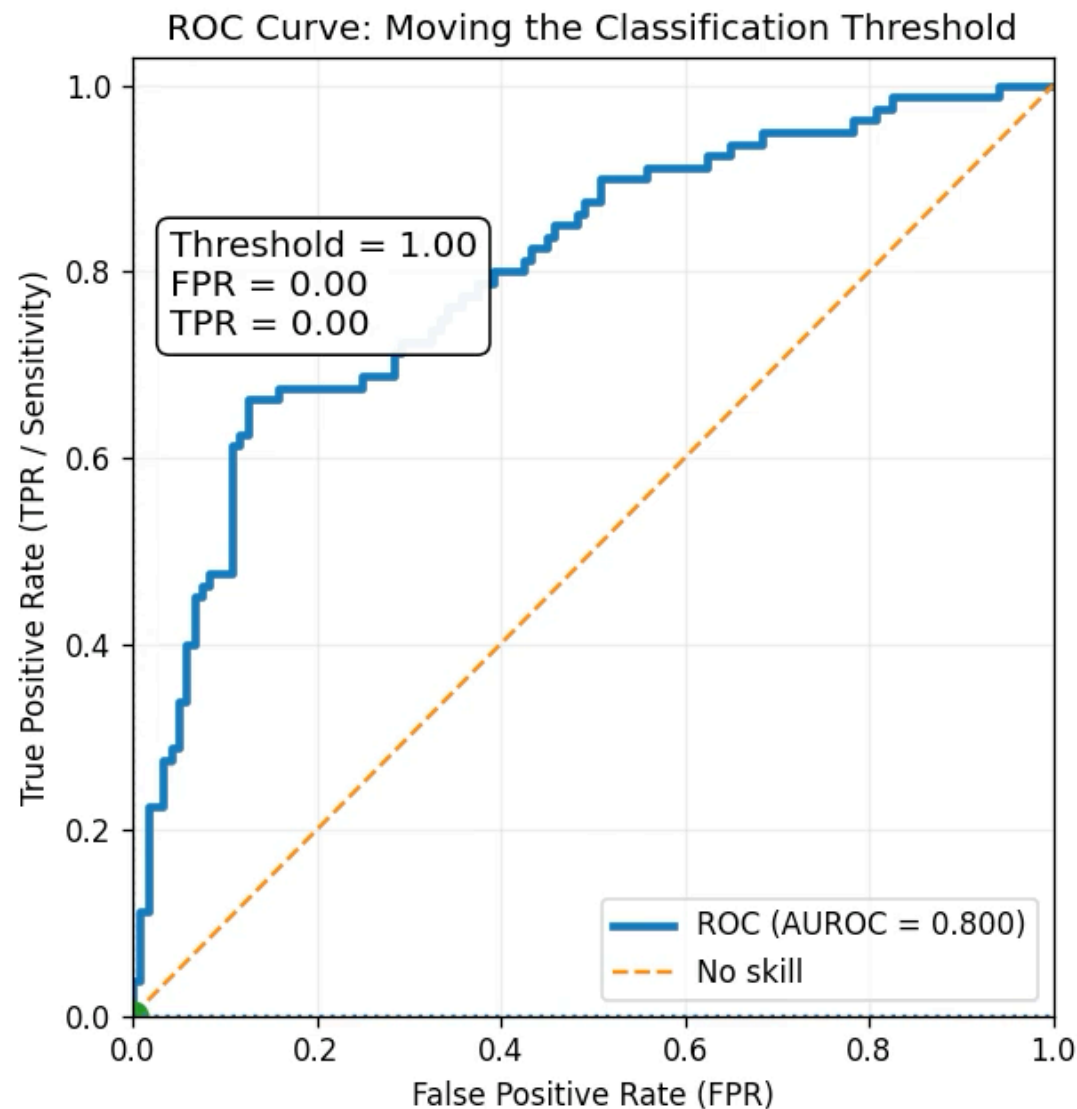
- $\text{TPR} = \text{Sensitivity} = \text{Recall}$
- $\text{FPR} = 1 - \text{Specificity}$

- Then plot: TPR vs FPR
- This produces the **Receiver Operating Characteristics (ROC)** curve. Each point on the ROC curve corresponds to one threshold t

ROC

	Pred. 0	Pred. 1	Total	Metric	Metric
Act. 0	TN	FP	TN+FP	$Sp = \frac{TN}{TN + FP}$	$FPR = \frac{FP}{FP + TN}$
Act. 1	FN	TP	FN+TP	$R = Se = TPR = \frac{TP}{TP + FN}$	
Total	TN+FN	FP+TP	TP+TN+FP+FN	$A = \frac{TP + TN}{TP + TN + FP + FN}$	
Metric		$P = \frac{TP}{TP + FP}$		$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	
Metric		$FDR = \frac{FP}{FP + TP}$			
Sp = Specificity; Se = Sensitivity; R = Recall; P = Precision; A = Accuracy					

ROC (animation)



Confusion Matrix

	Predicted -	Predicted +
Actual -	TN 120	FP 0
Actual +	FN 80	TP 0

Threshold 1.00 | Sensitivity 0.000 | Specificity 1.000 | Precision 0.000 | AUROC 0.800

ROC curve and AUROC stay fixed; moving the threshold changes the operating point and threshold-dependent metrics.

Contingency Table and ROC

ROC summarizes the behavior of many threshold-specific confusion matrices.

How to Interpret the ROC Curve?

- If the ROC curve is a diagonal line, i.e., $TPR = FPR$, the classifier is completely random.
- Consider a point on the diagonal line such as $FPR = 0.40$ and $TPR = 0.40$. The classifier is classifying 40% of actual positives as positives, but it is also classifying 40% of actual negatives as positives.

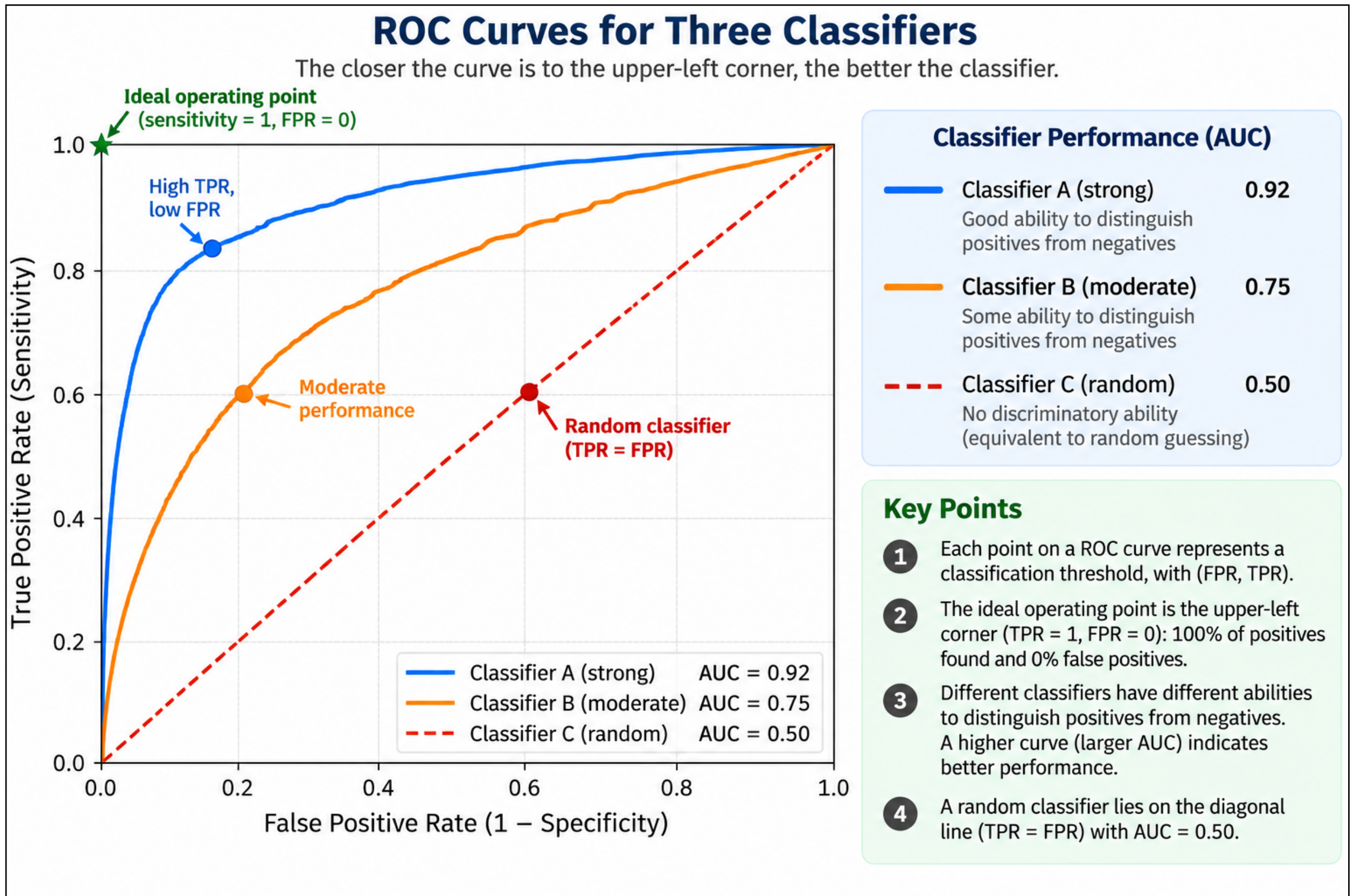
When TPR is increased to 0.50, FPR is also increased to 0.50.

- The classifier treats the two groups identically. Knowing the classifier's score provides no useful information for separating positive from negatives.
- The classifier has no ability whatsoever to distinguish positives from negatives. Its scores are unrelated to the true class.

What is AUC?

- AUC = Area Under the ROC Curve
- It is a single number summary of the ROC curve.
- Interpretation
 - AUC = 0.5: random guessing
 - AUC = 1.0: perfect prediction
 - AUC < 0.5: model is backwards.
- **AUROC is the probability that a randomly selected positive sample receives a higher model score than a randomly selected negative sample.**
- In logistic regression, model score is $p(Y = 1|X)$.

ROC and AUC



Precision-Recall

Imbalanced Dataset

Example: A dataset has 100 total subjects, among which 90 are positive and 10 are negative.

	Predicted Neg. (0)	Predicted Pos. (1)	Total	Metric
Actual Neg. (0)	5	5	10	$S = \frac{5}{10} = 0.5$
Actual Pos. (1)	10	80	90	$R = \frac{80}{90} = 0.88$
Total	15	85	100	$A = \frac{80 + 5}{100} = 0.85$
		$P = \frac{80}{85} = 0.94$		$F1 = 0.91$

Imbalanced Dataset

Example: A dataset has 100 total subjects, among which 9 are positive and 91 are negative.

	Pred. 0	Pred. 1	Total	Metric
Actual 0	90	1	91	$Sp = \frac{90}{90 + 1} = 0.99$
Actual 1	8	1	9	$R = Se = TPR = \frac{1}{9} = 0.11$
Total	98	2	100	$A = \frac{90 + 1}{100} = 0.91$
		$P = \frac{1}{2} = 0.5$		$F1 = 0.18$

misleading high accuracy

Imbalanced Dataset

Example: Suppose we have 1,000 patients, among which 990 do NOT have a disease (negative = 0) and 10 DO have the disease (positive = 1).

- 990 do NOT have the disease (negative = 0) and 10 DO have the disease (positive = 1).
- A dumb classifier can predict “No disease” for everyone. so:

$$TP = 0, TN = 990, FN = 10, FP = 0$$

- Accuracy will be:

$$Accuracy = \frac{TP + TN}{1000} = \frac{0 + 990}{1000} = 0.99$$

Accuracy is meaningless here, because the model is just exploiting the majority class.

- Recall will be:

$$Recall = \frac{TP}{TP + FN} = \frac{0}{0 + 10} = 0$$

It detects 0% of true positives.

Imbalanced Dataset

- One class is much rarer than the other.
- Metrics like accuracy can become misleading when one class dominates.
- Use the following instead:
 - Precision and recall
 - F1 score
 - Precision-Recall curve AUC

Precision-Recall Curve

- Precision-Recall Curve
 - x-axis: Recall
 - “Of all real positives, how many did we catch?”
 - y-axis: Precision
 - “Of all predicted positives, how many were correct?”

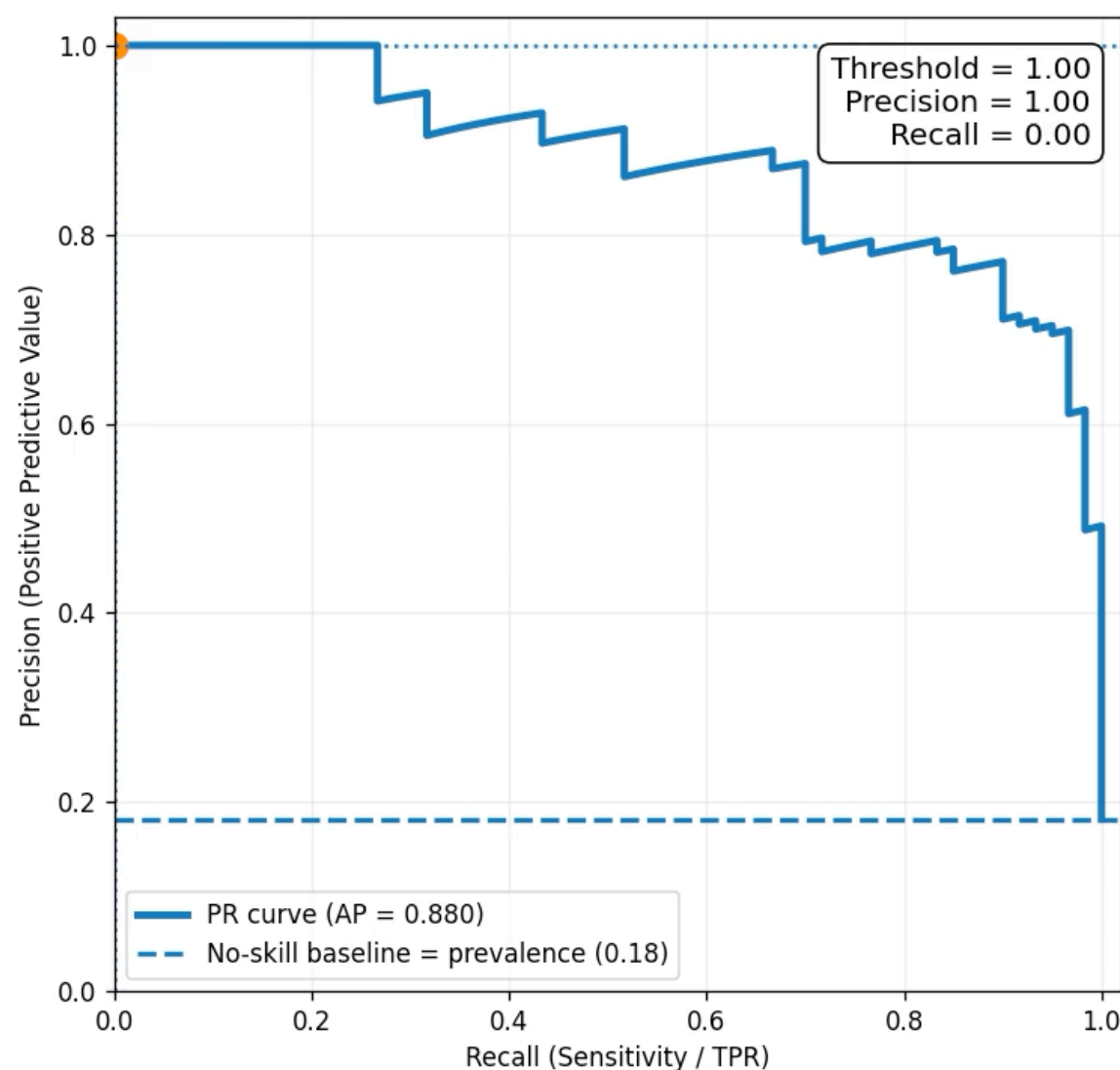
Precision-Recall Curve

	Pred. 0	Pred. 1	Total	Metric	Metric
Act. 0	TN	FP	TN+FP	$Sp = \frac{TN}{TN + FP}$	$FPR = \frac{FP}{FP + TN}$
Act. 1	FN	TP	FN+TP	$R = Se = TPR = \frac{TP}{TP + FN}$	
Total	TN+FN	FP+TP	TP+TN+FP+FN	$A = \frac{TP + TN}{TP + TN + FP + FN}$	
Metric		$P = \frac{TP}{TP + FP}$		$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	
Metric		$FDR = \frac{FP}{FP + TP}$			

Sp = Specificity; Se = Sensitivity; R = Recall; P = Precision; A = Accuracy

Precision-Recall Curve (animation)

Precision-Recall Curve: Moving the Classification Threshold



Classification Threshold



Threshold = 1.00

Confusion Matrix

	Pred -	Pred +
Actual -	270	0
Actual +	60	0

Performance Metrics

Precision (PPV)	1.000
Recall (Sensitivity)	0.000
Specificity (TNR)	1.000
False Positive Rate	0.000
F1 score	0.000
Accuracy	0.818

Average Precision 0.880

Lower threshold → generally higher recall, but precision may decrease. The PR curve and Average Precision remain fixed.

PR Curve: The Key Tradeoff

As you move along the curve:

- If you increase recall ...
 - TP increases (good)
 - FP also increases (bad)
 - So precision tends to drop.
- If you increase precision ...
 - FP decreases (good)
 - FN increases (bad)
 - So recall tends to drop.

What a good PR curve looks like?

A strong classifier has a curve that stays:

- High (precision close to 1) while also going
- far to the right (recall close to 1).

So the closer the curve is to the top-right, the better.

PR Curve: Baseline

- Unlike ROC curves, PR curves have a baseline that depends on your dataset.

- **Baseline precision = prevalence of positives**

If 5% of your dataset is positive, then the “random guessing” baseline is:

$$\text{Precision} = 0.05$$

- So if your PR curve is mostly above 0.05, you’re doing better than random.
- Many libraries report Average Precision (AP) that is essentially the area under the PR curve:
 - Higher AP = better model
 - AP is more meaningful than ROC-AUC for imbalanced data.

PR Curve: Practical Interpretation Example

If at some threshold, your model has:

- Recall = 0.90 and Precision = 0.20

This means:

- It catches 90% of true positive.
- But 80% of predicted positives are false alarms.

This might be fine in:

- disease screening, fraud detection, and safety alerts

But terrible in:

- expensive interventions and limited resource settings

How to Use the PR Curve?

You typically pick a threshold based on what matters more:

- If missing positives is costly:
→ choose a point with high recall
- If false positives are costly:
→ choose a point with high precision

ROC vs. PR

- Neither curve is universally “better.” They emphasize different aspects of performance.
- ROC is useful for understanding discrimination across both classes.
- PR is especially informative when the positive class is rare and finding positive cases is the primary concern.
- PR analysis focuses attention much more directly on:
When the model says “positive,” how often is it right, and how many true positives does it find?

Evaluation of Generalization Performance

Evaluation Protocol

The goal of evaluation is to estimate how well the trained model can be used on unseen data. Broadly speaking, evaluation methods include:

- **Hold-out validation**
- **K-fold cross-validation**
 - Leave-one-out cross-validation (LOOCV) is a special case of K-fold where K equals the number of samples.

Hold-Out Method

Procedure

- Split the dataset into two parts: training set and test set.
- Train a model on the training set
- Test the model on the test set

Why is it called hold-out?

Because you literally hold out some data and do not let the model see it during training. That held-out portion is used only for evaluation.

Typical split ratios

- 80% train / 20% test
- 70% train / 30% test

3-Way Hold-Out

Procedure

- Split the dataset into three parts: training set, validation set, and test set.
- Training set: used to train the model parameters
 - Fit the coefficients in linear regression
 - Fit the splits in a decision tree
- Validation set: used to choose:
 - hyperparameters (e.g., tree depth in decision tree)
 - model selection (which model family is best)
 - thresholds sometimes

Important: The validation set is used during development.

- Test set: used for final unbiased evaluation. It is used only once at the end to answer the questions: How well will this model perform on truly unseen data?

3-Way Hold-Out

Typical split ratios

- 60% train / 20% validation / 20% test
- 70% train / 15% validation / 15% test
- 80% train / 10% validation / 10% test (when data is limited)

Why Do We Need 3-Way Holdout?

Because if we tune hyperparameters using the test set, you're cheating, even if we don't mean to.

Example of “test set leakage”. You try

- $\text{max_depth} = 2 \rightarrow \text{test accuracy} = 0.80$
- $\text{max_depth} = 3 \rightarrow \text{test accuracy} = 0.84$
- $\text{max_depth} = 4 \rightarrow \text{test accuracy} = 0.86$
- $\text{max_depth} = 5 \rightarrow \text{test accuracy} = 0.83$

You choose $\text{max_depth} = 4$. But now your test set influenced your model choice. So your test accuracy is no longer a clean estimate.

Correct Workflow for 3-Way Split

- Step 1: Train on training set
Train the model with a chosen hyperparameter.
- Step 2: Evaluate on validation set
Pick the best hyperparameter based on validation performance.
- Step 3: Retrain using train + validation
Train again using more data.
- Step 4: Evaluate once on test set
Report the final performance.

Cross-Validation

- Cross-validation is a procedure for estimating how well a model will perform on unseen data by repeatedly training and testing it on different subsets of the dataset.
- Instead of relying on one train/test split, CV averages performance across multiple splits.

K-fold Cross-Validation

Procedure

- Split the dataset into K equal-sized folds
- For each fold $i = 1, 2, \dots, K$:
 - Train on the other $K - 1$ folds
 - Test on fold i .
- Compute the metric on each fold.
- Average the metrics.

K-fold Cross-Validation

Example: 5-fold CV

- Dataset split into 5 folds: A, B, C, D, E
- You run:
 - Train on B+C+D+E, $\rightarrow$ test on A
 - Train on A+C+D+E, $\rightarrow$ test on B
 - Train on A+B+D+E, $\rightarrow$ test on C
 - Train on A+B+C+E, $\rightarrow$ test on D
 - Train on A+B+C+D, $\rightarrow$ test on E
- Each sample is used:
 - 4 times in training
 - 1 time in testing

What does CV Output?

From the K-fold CV, you get:

- Fold scores: 0.81, 0.79, 0.83, 0.80, 0.82
- CV mean: 0.81
- CV std: 0.015

This is very useful because:

- mean = typical performance
- std = stability

What Values of K Should We Use?

Most common:

- $K = 5$ (good default, faster)
- $K = 10$ (more stable, slower)

General Rule:

- Larger $K \rightarrow$ more computation
- Smaller $K \rightarrow$ more variance

Leave-One-Out Cross-Validation

Core idea: If you have N samples, LOOCV does cross-validation:

- Hold out 1 sample as the test set
- Train the model on the other $N-1$ samples
- Evaluate on the held-out sample
- Repeat this N times, holding out a different sample each time
- Average the performance across all N trials

In one sentence:

LOOCV = k -fold cross validation where $K = N$.

Why Do We Use LOOCV?

- LOOCV uses almost all available data for training each time.
- That is extremely valuable when:
 - the dataset is small;
 - Every sample is expensive (e.g., clinical data, metabolomics samples, RNA-seq data)

LOOCV: Example

Suppose you have 5 samples as follows:

Sample ID	Label
1	0
2	0
3	1
4	1
5	1

LOOCV: Example

LOOCV runs 5 experiments:

- Fold 1
 - Train on samples: $\{2, 3, 4, 5\}$
 - Test on sample: $\{1\}$
- Fold 2
 - Train on samples: $\{1, 3, 4, 5\}$
 - Test on sample: $\{2\}$
- Fold 3
 - Train on samples: $\{1, 2, 4, 5\}$
 - Test on sample: $\{3\}$
- Fold 2
 - Train on samples: $\{1, 2, 3, 5\}$
 - Test on sample: $\{4\}$
- Fold 2
 - Train on samples: $\{1, 2, 3, 4\}$
 - Test on sample: $\{5\}$
- Then average the results.

LOO for Accuracy (Classification)

If each fold gives a correct or incorrect prediction:

Fold	Correct?
1	1
2	1
3	0
4	1
5	1

Then

$$\text{LOOCV Accuracy} = \frac{1 + 1 + 0 + 1 + 1}{5} = 0.80$$

Key Strengths of LOOCV

- Uses data efficiently: Each model trains on $N-1$ samples, so training sets are very large.
- Low bias estimate: Because training sets are almost full-sized, LOOCV tends to estimate performance close to what you'd get training on the full dataset.
- Great for very small datasets: If $N=20$, using 5-fold CV means training sets of size 16. LOOCV means training sets of size 19.

Key Weaknesses of LOOCV

- **Computationally expensive**

You train N models.

- $N=500 \rightarrow$ train 500 models
- $N=50,000 \rightarrow$ impossible in practice

- **Higher variance than k-fold (often!)**

Even though LOOCV uses more training data, the estimate can be noisy because:

- each test set is only 1 sample
- performance depends heavily on individual points (especially outliers)

- **Not great for unstable models**

- Decision trees, random forests (less so), neural nets can be sensitive.
- One sample removed may cause large model changes.

LOOCV vs K-Fold CV

Method	# Models Trained	Train Size	Test Size	Typical Use
Holdout	1	$\sim 70 - 80\%$	$\sim 20 - 30\%$	quick baseline
5-fold CV	5	80%	20%	most common
10-fold CV	10	90%	10%	common
LOOCV	N	N-1	1	very small datasets

When to use LOOCV?

LOOCV is best when:

- N is small (e.g., $N \leq 100$)
- training is fast (logistic regression, linear regression)
- you want maximum training data per fold

Avoid LOOCV when:

- N is large
- the model is slow
- the model is unstable
- the dataset has strong outliers

When to use LOOCV?

Simple guidelines.

Let n be the number of samples in your dataset:

- If $m \leq 20$, use LOOCV.
- if $20 < m \leq 1,000,000$, use k-fold cross validation.
- If $m \geq 1,000,000$, use hold-out cross validation. But if computational power is available, use k-fold cross validation ($k = 5$) if you want to squeeze that extra performance out of your model.

Nested Cross-Validation

OUTER CV LOOP (e.g., 5-fold): estimates generalization performance

Outer Fold 1

Outer Fold 2

Outer Fold 3

Outer Fold 4

Outer Fold 5

Repeat: choose 1 outer fold as TEST, remaining folds as TRAIN

INNER CV LOOP (e.g., 3-fold): chooses best hyperparameters

Inner 1

Inner 2

Inner 3

Pick hyperparameters with best mean INNER score

After nested CV:

- 1) Compare model families by OUTER mean score (select best model family)
- 2) Retrain/tune on FULL dataset to get final hyperparameters
- 3) Train ONE final model for deployment

Thank you!